

AERIS — CHECKPOINT 4

Transition from Geometry Engine → Aerodynamic Analysis Pipeline

0. Context and Intent

This project is the early-stage implementation of a **research-grade, AI-ready aircraft design platform**, focused on **Blended Wing Body (BWB) UAVs**.

The system is being built with the following long-term goals:

- * Deterministic, reproducible **geometry generation**
- * Large-scale dataset creation (10k+ designs)
- * ML-ready structured data (geometry → aero → stability)
- * Integration into a future **AI-driven MDAO + certification-aware pipeline**

We are **not building scripts**.

We are building a **scientific software system**.

1. What Has Been Built (Completed Phases)

1.1 CLI Backbone

* `aeris` CLI implemented using Typer

* Commands:

* `aeris dataset generate`

* `aeris dataset inspect`

✓ Single endpoint

✓ OS-agnostic

✓ Clean command surface

1.2 Geometry Engine (Core)

Fully modular pipeline:

...

sample → planform → sections → aerosandbox → export

...

Modules:

- * `geometry/sampling.py` → LHS sampling
- * `geometry/planform.py` → LE/TE generation
- * `geometry/sections.py` → section discretization
- * `geometry/aerosandbox\_adapter.py` → ASB conversion
- * `geometry/case.py` → orchestration

✓ Deterministic

✓ Seed-controlled

✓ No hidden randomness

1.3 Dataset System

Dataset structure:

...

data/datasets/<dataset_name>/

├─ geometry/

├─ metadata.csv

├─ failures.csv

├─ dataset_manifest.json

├─ logs/

├─ configs/

...

✓ Reproducible dataset creation

✓ Metadata includes:

* full design variables

* derived geometry

* artifact paths

✓ Failure tracking implemented

1.4 Metadata System

`dataset/metadata.py` defines:

- * full design vector logging

- * geometry summary stats

- * artifact traceability

✓ ML-ready tabular structure

✓ Explicit reproducibility fields (seeds, generator_id)

1.5 Determinism & Integrity Validation

Scripts:

- * `check\_architecture\_integrity.py`

- * `check\_metadata\_regeneration.py`

- * `check\_seed\_variation.py`

✓ Stable hashing

✓ Same input → same geometry

✓ Dataset reproducibility confirmed

1.6 Geometry Validation Layer (NEW — Critical)

Added:

...

geometry/validators.py

scripts/audit_dataset.py

...

Capabilities:

- * NaN / finite checks

- * monotonic span validation

- * chord positivity
- * TE > LE enforcement
- * span consistency
- * twist/dihedral sanity
- * dataset-level audit
- * regeneration validation
- ✓ Dataset is structurally valid
- ✓ Geometry pipeline is robust

1.7 Visualization System

Script:

'''

scripts/visualize_airplane.py

'''

Capabilities:

- * baseline reconstruction
- * dataset sample reconstruction
- * AeroSandbox `.draw()` visualization
- ✓ Baseline model recreated ****exactly****
- ✓ Architecture verified against legacy generator

2. Architectural Philosophy (Important)

This system enforces:

2.1 Deterministic Core

- * No randomness inside geometry
- * Sampling is the only stochastic layer

2.2 Strong Separation

| Layer | Responsibility |

| ----- | ----- |
| Sampling | Design space exploration |
| Geometry | Pure transformation |
| Dataset | Persistence |
| Validation | Trust layer |
| CLI | Interface |

2.3 Reproducibility First

Every geometry is:

- * traceable
- * regenerable
- * hashable

This is ****mandatory for ML + certification later****

3. What Was Just Achieved (Critical Milestone)

We proved:

- ✓ Architecture reproduces baseline geometry exactly
- ✓ Geometry pipeline is deterministic
- ✓ Dataset generation is stable
- ✓ Validation layer catches structural issues
- ✓ Reconstruction from metadata works

This means:

👉 The geometry engine is ****production-grade for dataset generation****

4. Current Limitation

The system currently produces:

- * geometry ✓
- * structure ✓
- * metadata ✓

But ****NO physics yet****

Missing:

- * CL / CD / Cm

- * stability derivatives

- * aerodynamic validity filtering

So right now:

``text

We have geometry data — not aerodynamic data

...

5. Next Phase — AERODYNAMIC PIPELINE

We now need to build:

...

geometry → aerodynamic analysis → dataset enrichment

...

6. Requirements for the Next Phase

The aerodynamic pipeline **MUST**:

6.1 Be Deterministic

- * same geometry → same aero results

6.2 Be Robust to Failures

- * non-converged cases handled

- * no crashes in batch runs

6.3 Be Dataset-Compatible

- * outputs added to metadata.csv

- * failure logging consistent

6.4 Be Physically Coherent

- * drag model consistency

* CL/CD meaningful

* no silent garbage values

7. Design Constraints (VERY IMPORTANT)

Avoid typical mistakes:

✗ mixing geometry + aero logic

✗ hidden solver randomness

✗ silent solver failures

✗ inconsistent drag definitions

✗ no validation of outputs

8. Immediate Objective

Design and implement:

🖱️ AERODYNAMIC ANALYSIS MODULE

Likely structure:

...

src/aeris/aero/

analysis.py

models.py

validation.py

...

And integrate into:

...

dataset_run.py

...

9. Key Design Questions (You MUST answer these)

1. Should we use:

- * AeroSandbox VLM?

- * AVL?

- * Both?

2. What outputs do we compute first?

- * CL

- * CD

- * Cm

- * L/D

3. At what flight condition?

- * fixed AoA?

- * trim condition?

- * sweep AoA?

4. How do we handle failures?

- * retry?

- * discard?

- * flag?

5. How do we ensure:

- * no NaNs

- * no unrealistic values

10. Expected Deliverable From You

You must:

1. Design the aero pipeline architecture

(not just code — structure + philosophy)

2. Define:

- * inputs

- * outputs

- * failure modes

3. Propose integration into dataset system

4. Provide first implementation plan

11. Tone & Level

Do NOT respond with:

- * generic explanations
- * beginner-level answers
- * high-level fluff

Respond as if:

👉 you are designing a system for a PhD + startup + certification pipeline

Be:

- * analytical
- * skeptical
- * precise

FINAL STATEMENT

The geometry engine is now stable.

We are moving into the ****most dangerous phase****:

👉 adding physics

If we get this wrong, the entire dataset becomes useless.

Design this carefully.